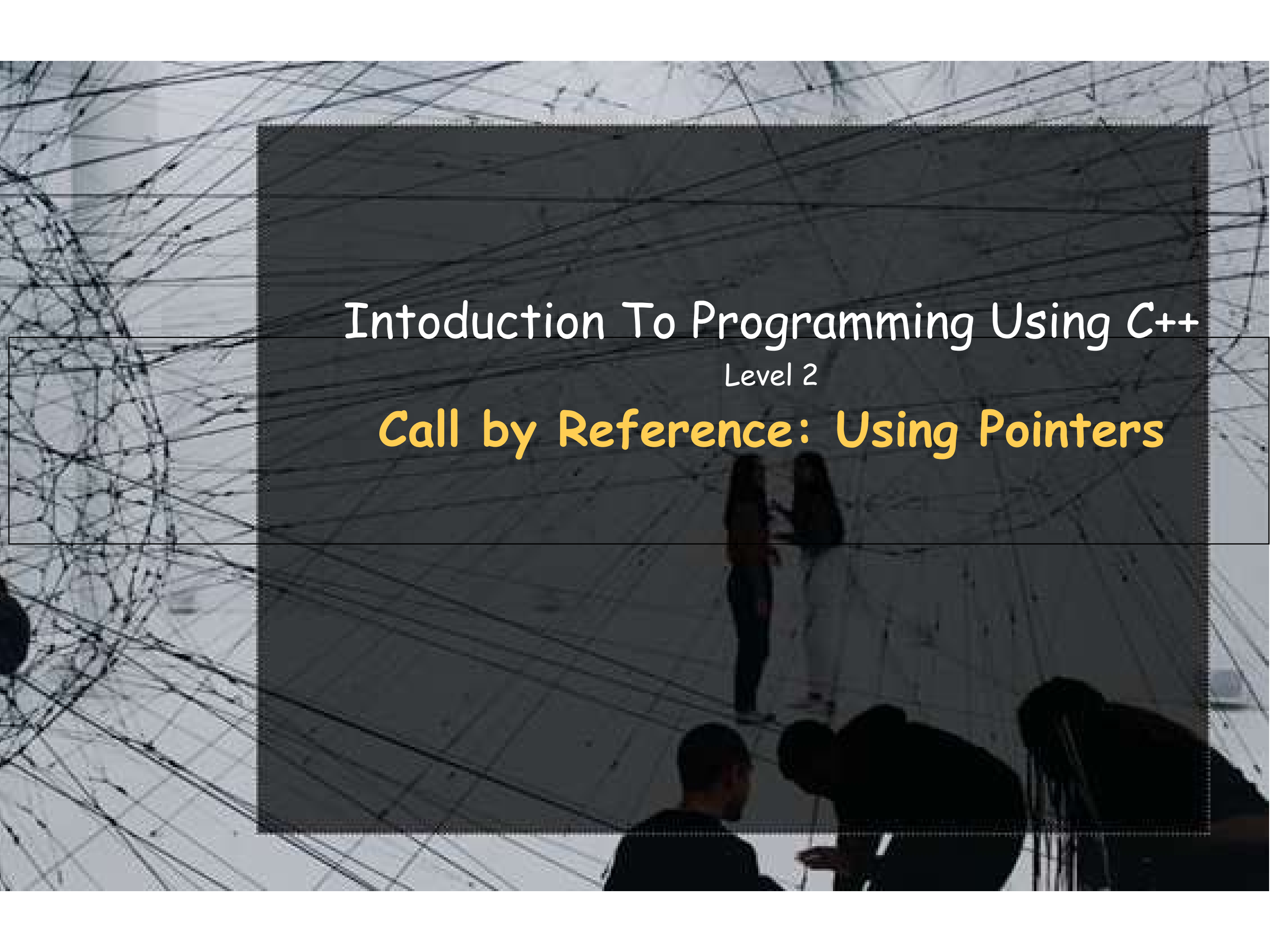


Intoduction To Programming Using C++ Level 2

Call by Reference: Using Pointers

The background of the slide is a photograph of a lecture hall. In the foreground, the silhouettes of several students are visible, some sitting and some standing. In the background, a large chalkboard is covered with faint, illegible lines of text, suggesting a lecture or classroom setting.

Introduction To Programming Using C++

Level 2

Call by Reference: Using Pointers

Definition

Call by Reference Using Pointers

 Call by Reference means allowing a function to modify the original variables instead of working with copies of their values.

 When pointers are used, the function receives the addresses of the original variables.

 The function can then use dereferencing (*) to access and modify the values stored at those addresses.

Explanation

Pointer

-  A pointer stores the address of a variable, not its value.

-  The * operator is used to access the value stored at the address held by the pointer.

Example

```
int A = 10;
```

```
int* P = &A;
```

-  P stores the address of A.

-  *P accesses the value of A.

Procedures

1. Create the original variables.
2. Create a function that receives pointers as parameters.
3. Pass the addresses of the original variables using &.
4. Inside the function, use * to access the original values.
5. Modify the values through the pointers.
6. Return to main().
7. Check the original variables and observe that their values have changed.

💡 Example

◆ Swap Using References

```
void Swap(int& Num1, int& Num2)
{
    int Temp = Num1;
    Num1 = Num2;
    Num2 = Temp;
}
int main()
{
    int A = 0, B = 1;
    Swap(A, B);
    return 0;
}
```

Note

Passing an Address

 To pass the address of *A*:
`&A`

Storing an Address

 To store the address in a pointer:
`P = &A;`

Accessing the Value

 To access the value through the pointer:
`*p`

Changing the Value

 To change the original variable through the pointer:
`*P = 20;`

⊘ Warning

■ Common Mistake

`*p = &A; // ✗ Incorrect`

■ This is incorrect because `*p` represents the value stored at the address held by `p`.

■ `&A` represents an address.

■ You cannot assign an address to a location that expects an int value.

■ Correct

`p = &A; // ✓ Stores the address of A`

`*p = A; // ✓ Copies the value of A`

`*p = 20; // ✓ Changes the value of A to 20`

◆ Remember:

`p` → Address

`*p` → Value

`&A` → Address of A

`A` → Value of A



Comparison

▨ Call by Reference

- ✓ The function receives references to the original variables.
- ✓ Example:
 `Swap(A, B);`
- ✓ Parameters use &:
 `void Swap(int& Num1, int& Num2)`
- ✓ Variables are accessed directly through the reference.
- ✓ No * is needed to access the value.
- ✓ The reference cannot be redirected to another variable.

▤ Call by Pointer

- ✓ The function receives pointers containing the addresses of the original variables.
- ✓ Example:
 `Swap(&A, &B);`
- ✓ Parameters use *:
 `void Swap(int* Num1, int* Num2)`
- ✓ * is needed to dereference the pointers and access the values.
- ✓ A pointer can be changed to point to another compatible variable.
- ✓ A pointer can also be nullptr, so the function can represent the absence of an object.

✦ Summary

- ✓ Reference → Another name (Alias) for the original variable.
- ✓ Pointer → A separate variable that stores the address of another variable.
- ✓ Call by Reference using references → Pass the variables directly to parameters declared with &.
- ✓ Call by Reference using pointers → Pass the addresses of variables using & and access their values using *.
- ✓ &A → Gets the address of A.
- ✓ P = &A → Stores the address of A in P.
- ✓ *P → Accesses the value stored at the address in P.
- ✓ *P = 20 → Changes the original variable to 20.
- ✓ Pointers allow a function to modify the original variables through their addresses.

Thank you 😊